MERN STACK

TRAINING TR-103 REPORT DAY 26

24 July 2026

Overview:

The twenty-sixth day of the MERN Stack training focused on integrating the Login API into the Career Connect project. We learned how to connect the frontend login form with the backend Login API, send user credentials using a POST request, handle the response received from the server, and redirect users according to their role. The session also helped in understanding how authentication can be used to provide different functionalities to different types of users such as students, recruiters, and administrators.

Learning Objectives:

- Understand the integration of frontend login with the backend Login API.
- Learn how to send login credentials from React to the server.
- Handle success and error responses from the API.
- Understand role-based access using usertype.
- Improve the overall login and authentication workflow.

Project : CareerConnect - Career Guidance and Management System

Career Connect Login Integration

The Login module in Career Connect is used to authenticate registered users before providing access to the application. The user enters their email and password through the login form, and these credentials are sent to the backend Login API for verification.

The backend checks the user details stored in MongoDB and returns an appropriate response based on whether the credentials are valid or invalid.

Features:

- Provides a secure login interface.
- Accepts email and password from the user.
- Sends credentials to the backend API.
- Validates user information from the database.
- Displays appropriate success and error messages.
- Redirects authenticated users to the required page.
- Supports different user roles using usertype.

Frontend Login Form

The React frontend provides input fields where users can enter their email and password. On clicking the Login button, an API request is sent to the backend.

The login form collects:

- Email
- Password

The entered information is sent to the server using a POST request.

API Integration

The frontend communicates with the backend Login API to authenticate the user.

Syntax:

```
axios.post("/api/login", {
    mail: email,
    password: password
});
```

Features:

• Connects React frontend with Node.js backend.

• Sends user credentials to the server.

• Receives authentication response from the API.

• Handles successful and unsuccessful login attempts.

• Helps establish communication between frontend and backend.

Response Handling

After sending the login request, the frontend receives a response from the backend.

If the login is successful, the response contains information such as:

• Status code

• Success message

• User ID

• User type

If the credentials are incorrect, an appropriate error message is displayed to the user.

Role-Based Navigation

Career Connect supports different types of users. The `usertype` received from the backend can be used to determine which section of the application the user should access.

For example:

- Student – Access to jobs, internships, applications, and career opportunities.
- Recruiter – Access to job posting and candidate management features.
- Admin – Access to overall system management and user-related functionality.

Based on the user's role, the application can navigate the user to the appropriate dashboard.

Navigation After Login

After successful authentication, the user is redirected to the relevant page using React Router.

Example:

```
navigate("/dashboard");
```

Role-based navigation can also be implemented according to the value of `usertype`.

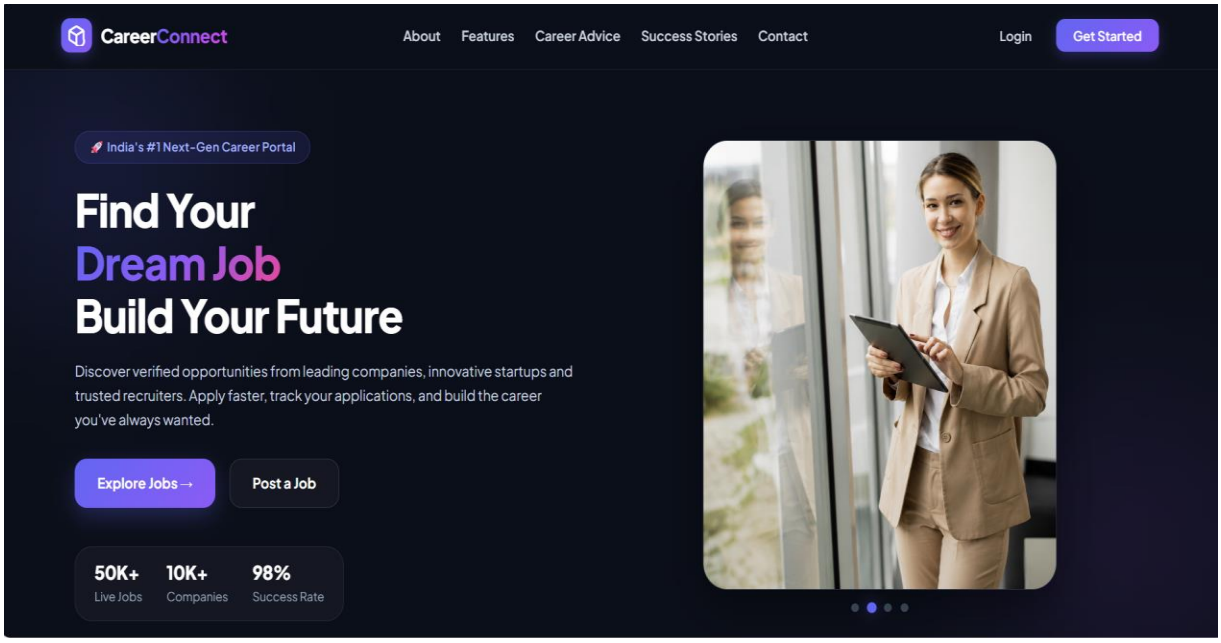
Working of Career Connect Login

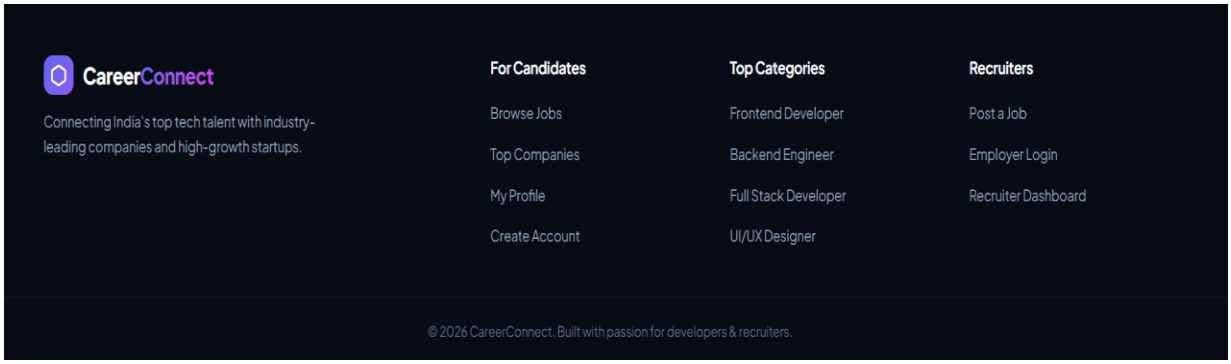
Steps:

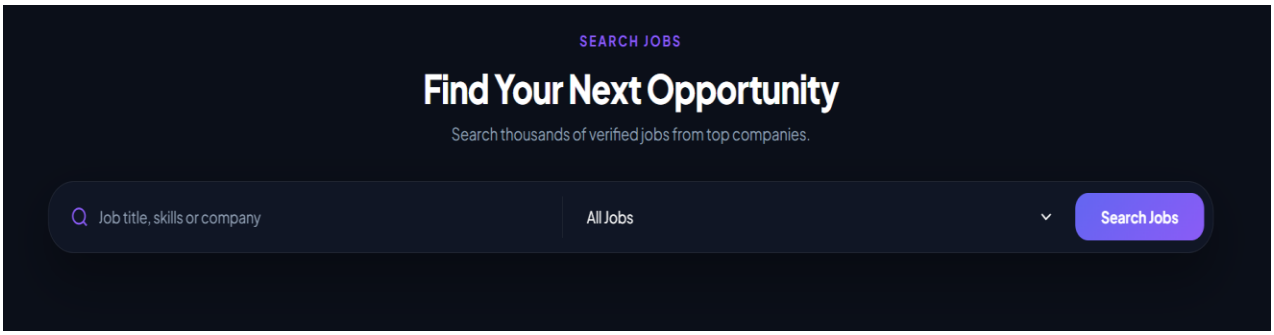
1. Create the login form using React.js.
2. Accept the user's email and password.
3. Send the entered credentials to the Login API using a POST request.
4. The backend searches for the user in MongoDB.
5. The server validates the entered password.
6. The backend sends a success or error response.
7. The frontend processes the received response.
8. If authentication is successful, the user's role is identified using `usertype`.
9. The user is redirected to the appropriate dashboard.
10. If authentication fails, an error message is displayed.

Conclusion:

By the end of Day 26, I gained a clear understanding of integrating a Login API with a MERN Stack application. I learned how to connect the React frontend with the backend, send user credentials through an API request, handle authentication responses, and redirect users based on their role. This practical implementation strengthened my understanding of authentication, API integration, React Router navigation, and role-based access control in the **Career Connect** project.






SEARCH JOBS
Find Your Next Opportunity
Search thousands of verified jobs from top companies.
Job title, skills or company
All Jobs
Search Jobs